

Arquitectura: Agente AI para Odoo

Microservicio autónomo · PDF → Word · Anthropic API

Este documento describe la arquitectura, las decisiones tecnológicas y el plan de construcción de un microservicio que conecta Odoo con un agente de inteligencia artificial. El agente procesa PDFs y genera documentos Word de forma completamente autónoma en producción, sin intervención manual.

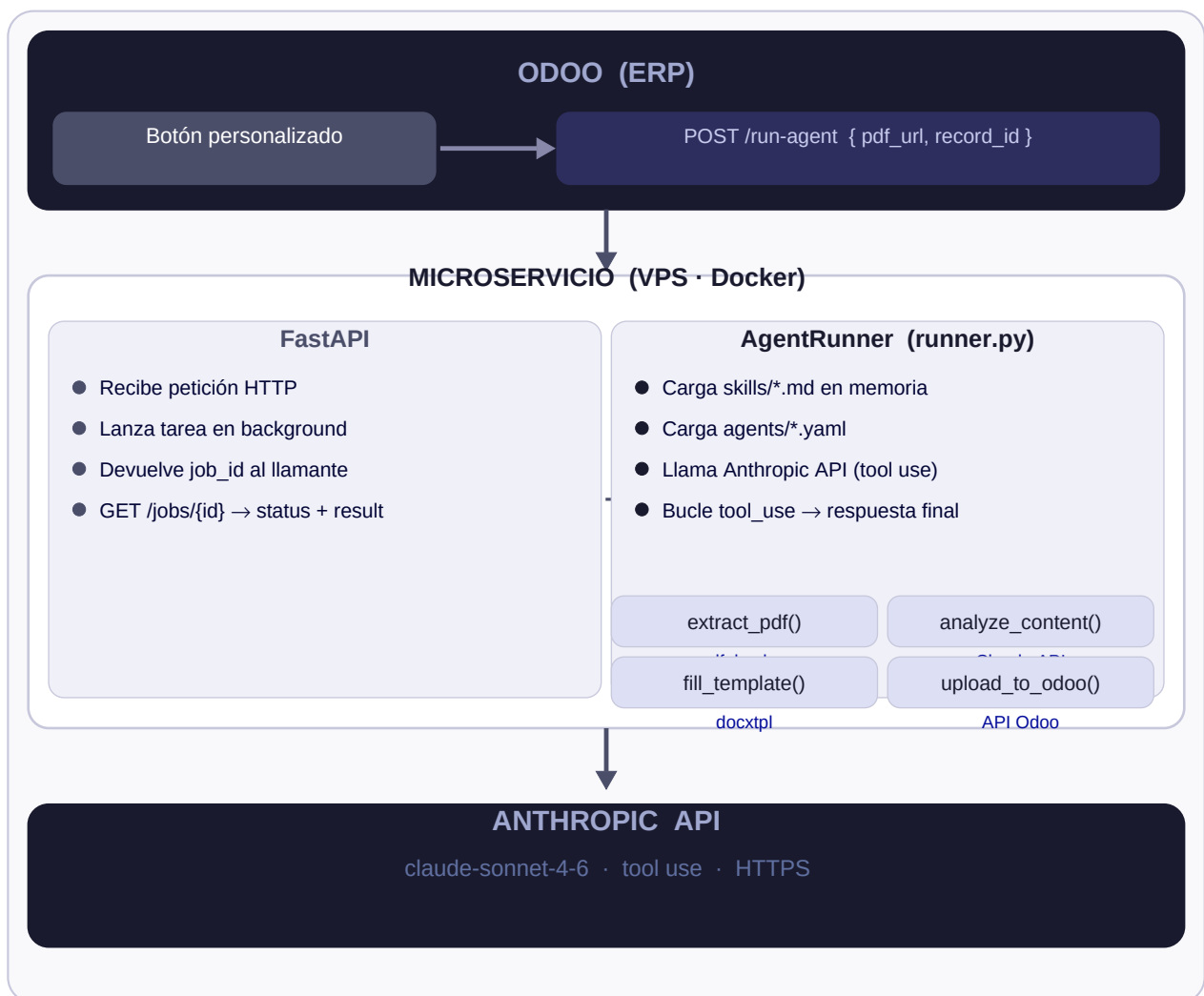
Preparado para el equipo · Junio 2026

1. ¿Qué queremos construir?

El sistema expone un botón personalizado en Odoo. Al pulsarlo, se llama a una API externa que lanza un agente de IA. El agente descarga un PDF desde Odoo, lo analiza mediante un conjunto de **skills** (ficheros Markdown), genera un documento Word siguiendo una plantilla y lo sube de vuelta a Odoo de forma completamente desatendida.

- El microservicio corre en un VPS dentro de un contenedor Docker.
- Los skills y la orquestación se definen en Markdown + YAML, editados con Claude Code.
- En producción no hay intervención manual: el agente opera de forma autónoma.
- El LLM es Claude vía Anthropic API directa (HTTPS), sin capas intermedias.

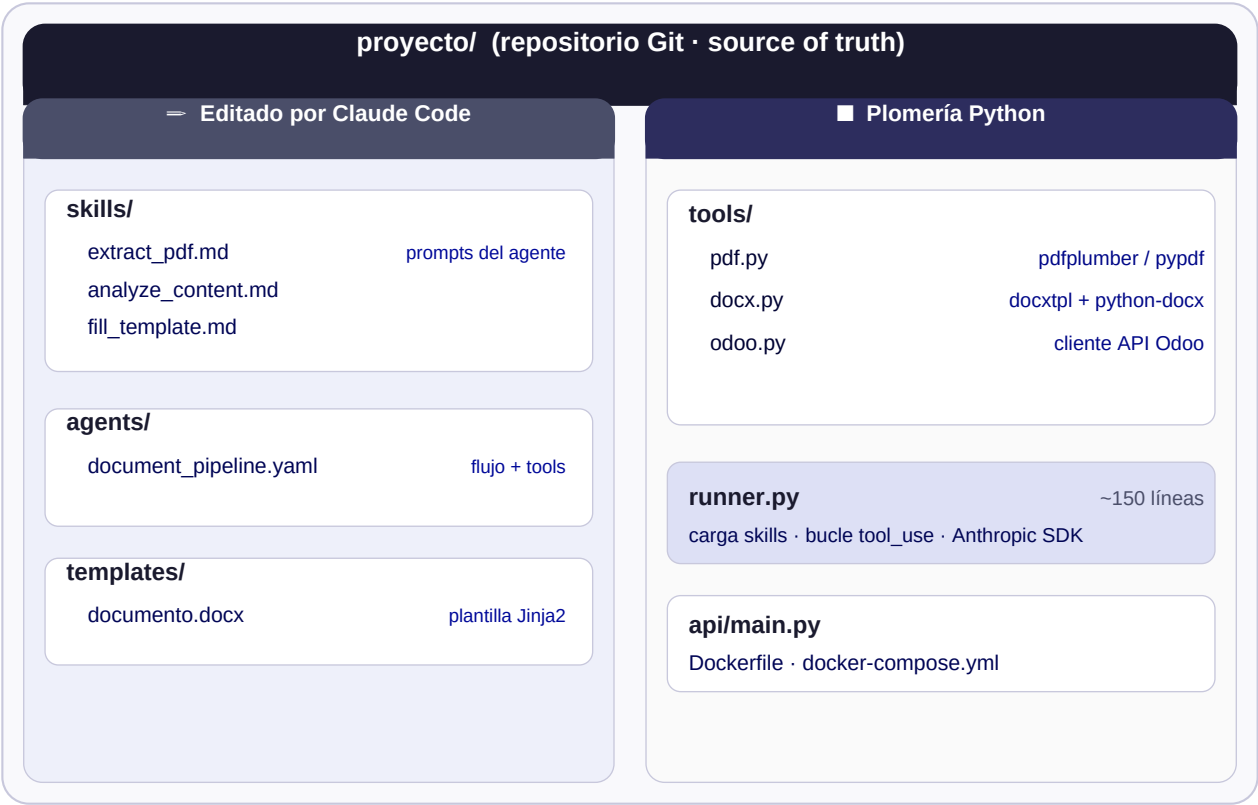
2. Flujo completo de extremo a extremo



3. Decisiones tecnológicas clave

Capa	Tecnología elegida	Razón
Runtime / API	Python + FastAPI	Ecosistema AI Python-first; async nativo; fácil containerización
Extracción PDF	pdfplumber / pypdf	Texto + tablas; OCR con pytesseract si el PDF es escaneado
Orquestación	Runner propio ~150 l	Skills en MD+YAML editables por Claude Code, sin traducción
LLM	Anthropic API directa	Sin intermediarios; tool use nativo; claude-sonnet-4-6
Generación Word	docxtpl + python-docx	Templates Jinja2 dentro del .docx; loops, condicionales
Contenedor	Docker + Compose	Skills baked-in en imagen; restart automático; sin orquestador
CI/CD	GitHub Actions	Push → build → deploy en VPS en ~2 minutos

4. Estructura del repositorio



Las carpetas **skills/** y **agents/** son ficheros de texto plano (Markdown y YAML). Claude Code los lee y edita directamente. El directorio **tools/** contiene funciones Python puras sin lógica de orquestación — solo ejecutan una acción concreta. El **runner.py** une todas las piezas; se escribe una vez y raramente cambia.

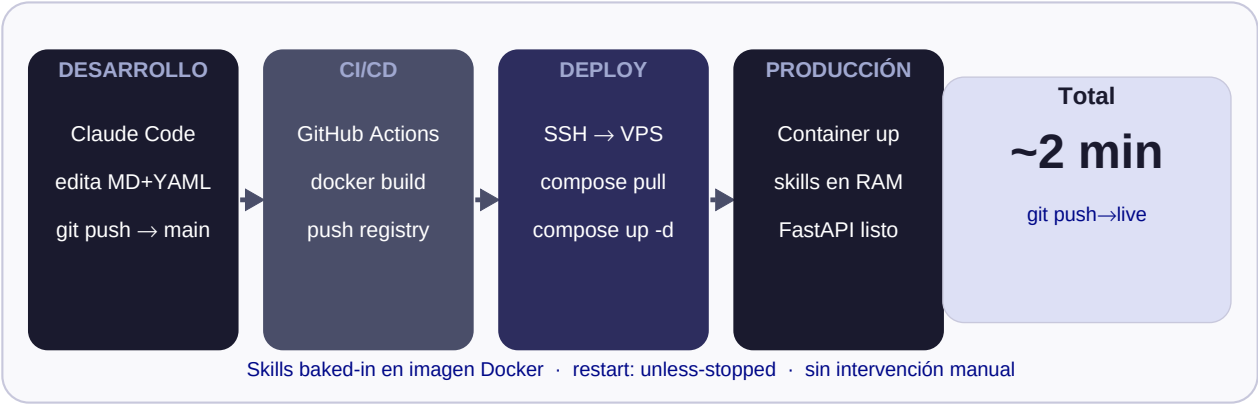
5. ¿Por qué no usamos LangChain o CrewAI?

Frameworks como LangChain o CrewAI son potentes, pero su lógica de agentes vive en clases Python con decoradores. Esto crea una **barrera de traducción**: Claude Code escribe el skill en Markdown, pero después hay que convertirlo manualmente al API del framework. Este proyecto elimina esa fricción por completo.

Criterio	LangChain / CrewAI	Runner propio
Lógica editable por Claude Code	Parcialmente (solo prompts)	Totalmente (MD + YAML)
Líneas de código propias	~50 (glue code)	~150 (runner completo)
Lock-in de framework	Alto	Ninguno
Curva de aprendizaje del equipo	Media-Alta	Baja
Dependencias en contenedor	Pesadas (50+ paquetes)	Mínimas
Integración con Anthropic API	Vía adaptadores langchain	SDK oficial directo

Si en el futuro se necesitan grafos de estado complejos para un subagente concreto, se puede integrar LangGraph solo para ese componente sin tocar el runner.

6. Ciclo de desarrollo y despliegue



Las skills y el YAML son editados localmente con Claude Code y subidos con **git push**. GitHub Actions construye la imagen Docker (los skills quedan baked-in como ficheros en la imagen), la sube al registry y hace deploy automático en el VPS. El contenedor arranca con **restart: unless-stopped** — no requiere supervisión.

7. Plan de construcción paso a paso

1	Scaffold del repositorio Estructura de carpetas, Dockerfile, docker-compose.yml y FastAPI skeleton. Variables de entorno: ANTHROPIC_API_KEY, ODOO_URL, ODOO_TOKEN.
---	--

2	Escribir el runner.py Motor ~150 líneas: carga skills/*.md y agents/*.yaml al arrancar, construye el system prompt, llama al SDK de Anthropic y gestiona el bucle tool_use.
3	Primer skill + primera tool Crear skills/extract_pdf.md y tools/pdf.py. Verificar el ciclo completo con un PDF de prueba antes de añadir más complejidad.
4	Pipeline CI/CD GitHub Actions: build → push a registry → SSH deploy en VPS. Validar que un git push actualiza el contenedor en menos de 3 min.
5	Integración con Odoo tools/odoo.py: descarga PDF, sube .docx. Endpoints: POST /run-agent (devuelve job_id) + GET /jobs/{id}.
6	Generación del documento Word Diseñar documento.docx con marcadores Jinja2. Implementar tools/docx.py con docxtpl. Probar con datos reales del PDF.
7	Skills restantes Claude Code escribe analyze_content.md y fill_template.md. Iterar sobre los prompts hasta obtener el output deseado.
8	QA end-to-end Prueba completa: botón Odoo → descarga PDF → Word generado → subida a Odoo. Ajustar skills en Markdown sin tocar el código Python.

8. Resumen visual del stack

